

Ściąga komend — Confluent Platform 8.3.0 / Apache Kafka 4.3.x

Zakres wersji: Confluent Platform 8.3.x zawiera Apache Kafka 4.3.x; pierwsze wydanie linii 8.3 ukazało się 17 czerwca 2026 r. Dokładne mapowanie to CP 8.3.0 → Kafka 4.3.0. ([Confluent — Supported Versions and Interoperability](#), [Confluent Platform 8.3 — Release Notes](#))

Stan dokumentacji: strony `kafka.apache.org/43/` są przypięte do Kafka 4.3. Strony Confluent oznaczone `platform/current` i `confluent-cli/current` opisują bieżącą linię 8.3.x / bieżące CLI i mogą obejmować poprawki wydane po 8.3.0. W miejscach, gdzie Confluent nie publikuje osobnej strony 8.3.0, zaznaczono konieczność sprawdzenia lokalnego `--help`.

0. Konwencje i szybki start

W dystrybucji Apache skrypty mają zwykle końcówkę `.sh` (`kafka-topics.sh`). Pakiety i obrazy Confluent mogą udostępniać równoległy wrapper bez końcówki (`kafka-topics`). Poniżej używana jest forma Apache; w CP można w razie potrzeby usunąć `.sh`. Wszystkie narzędzia znajdują się w katalogu `bin/` i po uruchomieniu bez argumentów lub z `--help` drukują dostępne opcje. ([Apache Kafka 4.3 — Basic Kafka Operations](#), [Confluent — Kafka CLI Tools](#))

```
export KAFKA_HOME=/opt/confluent
export PATH="$KAFKA_HOME/bin:$PATH"
export BOOTSTRAP='broker1.example.com:9092,broker2.example.com:9092'
export CONTROLLERS='controller1.example.com:9093,controller2.example.com:9093'
export ADMIN_CFG='/etc/kafka/admin.properties'
```

Zawsze sprawdź wersję rzeczywiście uruchamianego artefaktu:

```
kafka-topics.sh --version
kafka-storage.sh --help
kafka-metadata-quorum.sh --help
```

`--bootstrap-server`, `--bootstrap-controller`, **pliki properties**

- `--bootstrap-server`: łączy AdminClient z brokerem; standardowa opcja większości narzędzi.
- `--bootstrap-controller`: łączy bezpośrednio z listenerem kontrolera; użyteczne dla operacji KRaft.
- `--command-config <plik>`: przekazuje ustawienia klienta administracyjnego, np. TLS/SASL.
- Stosuj co najmniej 2–3 adresy bootstrap, rozdzielone przecinkami; nie jest to pełna lista członków klastra.

Źródła składni: [Apache Kafka 4.3 — KRaft](#) i [Confluent — Kafka CLI Tools](#).

1. KRaft i metadata quorum

1.1. kafka-storage.sh random-uuid

Cel: generuje losowy identyfikator w formacie UUID używany jako `cluster.id` albo `directory.id`.

```
CLUSTER_ID="$(kafka-storage.sh random-uuid)"
echo "$CLUSTER_ID"
```

Najważniejsze:

- jeden `CLUSTER_ID` musi być użyty do sformatowania wszystkich węzłów tego samego klastra;
- komenda generuje również poprawny identyfikator katalogu do `--initial-controllers`;
- nie generuj nowego `cluster.id` dla istniejącego klastra.

Źródło: [Apache Kafka 4.3 — Provisioning Nodes](#).

1.2. kafka-storage.sh format --standalone

Cel: formatuje pierwszy kontroler **nowego** dynamicznego quorum jako pojedynczego votera.

```
kafka-storage.sh format \
  --cluster-id "$CLUSTER_ID" \
  --standalone \
  --config /etc/kafka/controller.properties
```

Komenda tworzy `meta.properties` z losowym `directory.id` oraz początkowy snapshot KRaft z `KRaftVersionRecord` i jednoelementowym `VotersRecord`. Nie jest to metoda odzyskiwania istniejącego metadata logu. ([Apache Kafka 4.3 — Bootstrap a Standalone Controller](#))

Najważniejsze flagi:

Flaga	Znaczenie
<code>-t, --cluster-id <ID></code>	ID klastra
<code>-c, --config <plik></code>	konfiguracja węzła
<code>-s, --standalone</code>	nowy, jednoelementowy dynamiczny quorum
<code>-g, --ignore-formatted</code>	pomija już sformatowane katalogi zamiast kończyć błędem
<code>-r, --release-version <wersja></code>	początkowe poziomy feature dla wydania
<code>-f, --feature feature=level</code>	jawny poziom pojedynczej funkcji
<code>-S, --add-scram ...</code>	dodaje początkowe poświadczenie SCRAM do metadata logu

Pełna składnia linii `current`, weryfikowana dla narzędzi dostarczanych z CP 8.3.x: [Confluent — kafka-storage.sh](#). Dla dokładnego artefaktu 8.3.0 zalecane jest `kafka-storage.sh` format `--help`.

1.3. `kafka-storage.sh` format `--no-initial-controllers`

Cel: formatuje brokera lub nowy kontroler dołączany do istniejącego dynamicznego quorum, bez tworzenia początkowej topologii voterów.

```
kafka-storage.sh format \  
  --cluster-id "$CLUSTER_ID" \  
  --config /etc/kafka/controller-new.properties \  
  --no-initial-controllers
```

Po sformatowaniu i uruchomieniu nowy kontroler pojawia się jako observer. Najpierw czekaj na synchronizację (`describe --replication`), a dopiero potem wykonaj `add-controller`. Źródło: [Apache Kafka 4.3 — Formatting Brokers and New Controllers](#).

1.4. Alternatywa: bootstrap wielu kontrolerów

Zalecana ścieżka 4.3 to bootstrap jednego votera i dynamiczne dodanie pozostałych, ale nowy klaster można także sformatować z pełnym początkowym składem:

```
C0_DIR_ID="$(kafka-storage.sh random-uuid)"  
C1_DIR_ID="$(kafka-storage.sh random-uuid)"  
C2_DIR_ID="$(kafka-storage.sh random-uuid)"  
  
kafka-storage.sh format \  
  --cluster-id "$CLUSTER_ID" \  
  --initial-controllers \  
  "0@controller-0.example.com:9093:${C0_DIR_ID},1@controller-1.example.com:9093:${C1_DIR_ID},2@controller-2.example.com:9093:${C2_DIR_ID}" \  
  --config /etc/kafka/controller.properties
```

Dokładnie ta sama wartość `--initial-controllers` musi zostać użyta przy formatowaniu wszystkich początkowych kontrolerów. Format wpisu to `node.id@host:port:directory.id`. Źródło: [Apache Kafka 4.3 — Bootstrap with Multiple Controllers](#).

1.5. `kafka-metadata-quorum.sh` describe `--status`

Cel: syntetyczny stan quorum: lider, epoch, high watermark, maksymalny lag, votery i observery.

```
kafka-metadata-quorum.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  describe --status
```

Bezpośrednio przez listener kontrolera:

```
kafka-metadata-quorum.sh \  
  --bootstrap-controller "$CONTROLLERS" \  
  --command-config "$ADMIN_CFG" \  
  describe --status
```

Kluczowe pola: LeaderId, LeaderEpoch, HighWatermark, MaxFollowerLag, MaxFollowerLagTimeMs, CurrentVoters, CurrentObservers. Źródło i przykładowy output: [Apache Kafka 4.3 — Metadata Quorum Tool](#).

1.6. kafka-metadata-quorum.sh describe --replication

Cel: stan replikacji każdego votera/observera, w tym LogEndOffset i opóźnienie wobec lidera.

```
kafka-metadata-quorum.sh \  
  --bootstrap-controller "$CONTROLLERS" \  
  --command-config "$ADMIN_CFG" \  
  describe --replication
```

Używaj przed promocją nowego kontrolera. Observer powinien dogonić aktywnego lidera; nie promuj go tylko dlatego, że proces działa. Procedura jest opisana w [Apache Kafka 4.3 — Add New Controller](#).

1.7. kafka-metadata-quorum.sh add-controller

Cel: promuje zsynchronizowany kontroler-observer do votera w dynamicznym quorum (kraft.version >= 1).

Na nowym kontrolerze konfiguracja przekazana w --command-config musi zawierać jego właściwości kontrolera (node.id, listener/endpoints, metadata.log.dir) oraz — jeśli wymagane — właściwości uwierzytelnienia AdminClient:

```
kafka-metadata-quorum.sh \  
  --bootstrap-controller "$CONTROLLERS" \  
  --command-config /etc/kafka/controller-new.properties \  
  add-controller
```

Wariant przez brokera:

```
kafka-metadata-quorum.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config /etc/kafka/controller-new.properties \  
  add-controller
```

Źródło: [Apache Kafka 4.3 — Add New Controller](#).

1.8. kafka-metadata-quorum.sh remove-controller

Cel: usuwa votera z dynamicznego quorum. Usuń membership **przed** zatrzymaniem procesu.

```
kafka-metadata-quorum.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  remove-controller \  
  --controller-id 3 \  
  --controller-directory-id 'AbCdEf1234567890xyz'
```

Obie wartości muszą identyfikować ten sam członek quorum. `directory.id` odczytasz z `meta.properties` albo `describe --status`. Źródło: [Apache Kafka 4.3 — Remove Controller](#).

1.9. kafka-metadata-recovery reconfig log-length

Cel: offline odczyt metadata epoch i log end offset z zatrzymanego kontrolera podczas odzyskiwania po utracie quorum.

```
# Kafka/KRaft na tym wolumenie musi być zatrzymana.  
kafka-metadata-recovery reconfig log-length \  
  --metadata-log-dir /var/lib/kafka/metadata
```

Status dokumentacji: to narzędzie specyficzne dla Confluent, nie standardowy skrypt Apache Kafka. Publiczna, dokładna składnia jest obecnie opisana w procedurze Disaster Recovery dla Confluent for Kubernetes, a nie na osobnej stronie CP 8.3.0 dla RPM/TAR. Dla instalacji nie-CFK sprawdź lokalnie:

```
kafka-metadata-recovery --help  
kafka-metadata-recovery reconfig --help  
kafka-metadata-recovery reconfig log-length --help
```

Źródło: [Confluent — Disaster Recovery for Multi-Region KRaft Clusters](#).

1.10. kafka-metadata-recovery reconfig force-standalone

Cel: nieodwracalna, offline przebudowa zachowanego metadata logu do jednoelementowego dynamicznego quorum po utracie starego quorum.

```
# Najpierw zatrzymaj wszystkie ocalałe kontrolery i wykonaj snapshot storage.  
kafka-metadata-recovery reconfig force-standalone \  
  --config /etc/kafka/controller.properties
```

Krytyczne ostrzeżenia:

1. wybierz seed według najwyższego metadata epoch; LEO jest dopiero tie-breakerem;
2. wykonaj polecenie dokładnie raz, wyłącznie na seedzie;

3. nie uruchamiaj ponownie po przerwaniu lub niejednoznacznym błędzie – skontaktuj się z Confluent Support;
4. `kafka-storage format --standalone` nie jest zamiennikiem tej operacji;
5. utrata kontrolerów posiadających dawną większość może oznaczać utratę wcześniej zatwierdzonych metadanych, których nie miał seed.

Status dokumentacji: podobnie jak `log-length`, publiczny runbook dotyczy CFK; dla czystego CP 8.3.0 należy potwierdzić obecność i help lokalnego binarium oraz prowadzić operację z Confluent Support. Źródło: [Confluent — manual quorum-loss recovery](#).

1.11. `kafka-features.sh describe`

Cel: pokazuje obsługiwane i sfinalizowane poziomy funkcji, przede wszystkim `metadata.version` i `kraft.version`.

```
kafka-features.sh \
  --bootstrap-controller "$CONTROLLERS" \
  --command-config "$ADMIN_CFG" \
  describe
```

Interpretacja:

- `kraft.version` brak lub `FinalizedVersionLevel: 0` → quorum statyczny;
- `kraft.version >= 1` → dynamiczny membership i `add-controller / remove-controller`;
- dla CP 8.3.x docelowa `metadata.version` to 4.3-IV0.

Źródła: [Apache Kafka 4.3 — Describe KRaft Version](#) oraz [Confluent — Upgrade CP](#).

1.12. Upgrade `metadata.version` i `kraft.version`

Po rolling upgrade wszystkich brokerów i kontrolerów do 4.3:

```
# Najpierw zobacz mapowanie bez zmiany:
kafka-features.sh \
  --bootstrap-server "$BOOTSTRAP" \
  version-mapping \
  --release-version 4.3

# Finalizacja całego zestawu funkcji dla wydania 4.3:
kafka-features.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" \
  upgrade --release-version 4.3
```

Upgrade tylko funkcji KRaft z quorum statycznego do dynamicznego:

```
kafka-features.sh \
  --bootstrap-controller "$CONTROLLERS" \
```


Snapshot .checkpoint:

```
kafka-dump-log.sh \  
  --cluster-metadata-decoder \  
  --files /var/lib/kafka/metadata/__cluster_metadata-0/0000000000000000000100-000000000
```

Pracuj na kopii katalogu, szczególnie podczas incydentu. Narzędzie dekoduje pliki, ale nie naprawia quorum. Źródło: [Apache Kafka 4.3 — Dump Log Tool](#).

2. Tematy, partycje i repliki

Poniższa składnia jest opisana dla narzędzi bieżącej linii Confluent oraz Kafka 4.3; do operacji na topicach Confluent publikuje też osobny przewodnik [Topic Operations](#).

2.1. kafka-topics.sh --create

```
kafka-topics.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --create \  
  --topic orders \  
  --partitions 12 \  
  --replication-factor 3 \  
  --config min.insync.replicas=2 \  
  --config cleanup.policy=delete \  
  --if-not-exists
```

Najważniejsze: `--topic`, `--partitions`, `--replication-factor`, wielokrotne `--config`, `--replica-assignment`, `--if-not-exists`. Konfiguracje można podać przy `--create`; późniejsze zmiany wykonuje się przez `kafka-configs`, nie przez stare `--alter --config`. Źródła: [Apache Kafka 4.3 — Adding and removing topics](#) i [Confluent — kafka-topics.sh](#).

2.2. Listowanie

```
kafka-topics.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --list  
  
kafka-topics.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --list --exclude-internal
```

Źródło: [Confluent — kafka-topics.sh](#).

2.3. Opis tematu i filtry diagnostyczne

```
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" --describe --topic orders
```

```
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe --under-replicated-partitions
```

```
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe --unavailable-partitions
```

```
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe --under-min-isr-partitions
```

```
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe --at-min-isr-partitions
```

Przydatne flagi: `--topic <regex>`, `--topic-id`, `--topics-with-overrides`, `--exclude-internal`.

Źródło: [Confluent — kafka-topics.sh usage](#).

2.4. Zwiększanie liczby partycji

```
kafka-topics.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --alter \  
  --topic orders \  
  --partitions 24
```

Liczbę partycji można tylko zwiększyć. Dla rekordów z kluczem zmiana liczby partycji zmienia wynik partycjonowania i może naruszyć oczekiwane uporządkowanie per key. Źródło: [Apache Kafka 4.3 — Modifying topics](#).

2.5. Usuwanie tematu

```
kafka-topics.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --delete \  
  --topic orders \  
  --if-exists
```

Usunięcie jest logicznie nieodwracalne; fizyczne pliki mogą zniknąć asynchronicznie. Źródło: [Apache Kafka 4.3 — Deleting a topic](#).

2.6. Reassignment partycji — generate / execute / verify

Plik topics-to-move.json:

```
{"topics":[{"topic":"orders"}, {"topic":"payments"}], "version":1}
```

Generowanie planu:

```
kafka-reassign-partitions.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" \
  --topics-to-move-json-file topics-to-move.json \
  --broker-list '1,2,3,4' \
  --generate
```

Zapisz zarówno Current partition replica assignment, jak i Proposed partition reassignment configuration. Następnie umieść wybrany plan w reassignment.json:

```
{
  "version": 1,
  "partitions": [
    {"topic":"orders","partition":0,"replicas":[2,3,4],"log_dirs":["any","any","any"]}
  ]
}
```

Uruchomienie i walidacja:

```
kafka-reassign-partitions.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" \
  --reassignment-json-file reassignment.json \
  --execute \
  --throttle 50000000 \
  --replica-alter-log-dirs-throttle 100000000
```

```
kafka-reassign-partitions.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" \
  --reassignment-json-file reassignment.json \
  --verify
```

Operacje dodatkowe:

```
# Lista aktywnych reassignmentów
kafka-reassign-partitions.sh --bootstrap-server "$BOOTSTRAP" --list

# Anulowanie aktywnego planu
kafka-reassign-partitions.sh \
  --bootstrap-controller "$CONTROLLERS" \
```

```
--reassignment-json-file reassignment.json \  
--cancel
```

Najważniejsze flagi: `--generate`, `--execute`, `--verify`, `--list`, `--cancel`, `--additional`, `--preserve-throttles`, `--disable-rack-aware`, `--throttle`, `--replica-alter-log-dirs-throttle`. `--verify` usuwa throttle po zakończeniu, o ile nie użyto `--preserve-throttles`. Źródła: [Apache Kafka 4.3 — Expanding your cluster](#) i [Confluent — kafka-reassign-partitions.sh](#).

2.7. kafka-replica-verification.sh

Cel: ciągle porównanie danych replik tematów pasujących do regexu.

```
kafka-replica-verification.sh \  
--broker-list 'broker1:9092,broker2:9092,broker3:9092' \  
--topics-include 'orders|payments' \  
--report-interval-ms 30000
```

Opcje: `--fetch-size`, `--max-wait-ms`, `--time` (-1 latest, -2 earliest), `--topics-include`. W **Kafka/CP 4.3 narzędzie jest oznaczone jako deprecated i może zniknąć w kolejnym major release**. Źródło: [Confluent — kafka-replica-verification.sh](#).

3. Konfiguracja dynamiczna — kafka-configs.sh

3.1. Wzorzec describe

```
kafka-configs.sh \  
--bootstrap-server "$BOOTSTRAP" \  
--command-config "$ADMIN_CFG" \  
--describe \  
--entity-type <topics|brokers|clients|users|ips> \  
--entity-name <nazwa>
```

`--all` pokazuje także dostępne ustawienia statyczne. `--entity-default` wybiera wartość domyślną danego typu. Do kontrolerów można łączyć się przez `--bootstrap-controller`, bez równoczesnego `--bootstrap-server`. Źródło: [Confluent — kafka-configs.sh](#).

3.2. Tematy

```
# Odczyt  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
--describe --entity-type topics --entity-name orders  
  
# Dodanie/zmiana  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
--alter --entity-type topics --entity-name orders \  
--add-config 'retention.ms=604800000,min.insync.replicas=2'  
  
# Przywrócenie wartości dziedziczonej
```

```
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --alter --entity-type topics --entity-name orders \  
  --delete-config 'retention.ms'
```

Źródło: [Apache Kafka 4.3 — Modifying topic configuration](#).

3.3. Broker lub domyślna konfiguracja klastra

```
# Konkretny broker  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe --entity-type brokers --entity-name 1 --all  
  
# Dynamiczny default dla wszystkich brokerów  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --alter --entity-type brokers --entity-default \  
  --add-config 'num.io.threads=16'  
  
# Lista z przecinkami musi być ujęta w nawiasy kwadratowe  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --alter --entity-type brokers --entity-default \  
  --add-config 'max.connections.per.ip.overrides=[host1:50,host2:9]'
```

Nie wszystkie broker configs są dynamiczne; sprawdź tryb aktualizacji danej właściwości (read-only, per-broker, cluster-wide) w [Confluent — Broker Configuration Reference](#). Składnia narzędzia: [Confluent — kafka-configs.sh](#).

3.4. Quota klienta

```
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --alter \  
  --entity-type clients --entity-name billing-producer \  
  --add-config 'producer_byte_rate=10485760,request_percentage=50'  
  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe \  
  --entity-type clients --entity-name billing-producer
```

Typowe klucze: producer_byte_rate, consumer_byte_rate, request_percentage, controller_mutation_rate. Źródło: [Confluent — kafka-configs.sh entity types](#).

3.5. Użytkownik: SCRAM i quota

```
# Utworzenie/zmiana poświadczenia SCRAM  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --alter \  
  --entity-type users --entity-name alice \  
  --add-config 'SCRAM-SHA-512=[iterations=8192,password=ChangeMe]'  
  
# Odczyt  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe --entity-type users --entity-name alice
```

```
--describe \  
--entity-type users --entity-name alice  
  
# Usunięcie mechanizmu  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
--alter \  
--entity-type users --entity-name alice \  
--delete-config 'SCRAM-SHA-512'
```

Typ `users` obsługuje SCRAM-SHA-256, SCRAM-SHA-512 i quota użytkownika. Źródła: [Apache Kafka 4.3 — SASL/SCRAM](#) oraz [Confluent — kafka-configs.sh](#).

3.6. Quota złożona użytkownik + klient

```
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
--alter \  
--entity-type users --entity-name alice \  
--entity-type clients --entity-name billing-producer \  
--add-config 'producer_byte_rate=5242880'
```

`users` i `clients` można podać razem, aby ustawić quota dla danego `client-id` konkretnego użytkownika. Źródło: [Confluent — kafka-configs.sh usage](#).

3.7. IP

```
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
--alter \  
--entity-type ips --entity-name 192.0.2.10 \  
--add-config 'connection_creation_rate=20'  
  
kafka-configs.sh --bootstrap-server "$BOOTSTRAP" \  
--describe \  
--entity-type ips --entity-name 192.0.2.10
```

Typ encji to `ips` (liczba mnoga), a główny klucz to `connection_creation_rate`. Źródło: [Confluent — kafka-configs.sh entity types](#).

4. Producent, konsument i grupy

4.1. kafka-console-producer.sh

```
kafka-console-producer.sh \  
--bootstrap-server "$BOOTSTRAP" \  
--command-config /etc/kafka/client.properties \  
--topic orders
```

Klucz i wartość:

```
kafka-console-producer.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config /etc/kafka/client.properties \
  --topic orders \
  --reader-property parse.key=true \
  --reader-property key.separator=:

# stdin:
# order-123:{"status":"NEW"}
```

W 4.3 preferowane są `--command-config`, `--command-property`, `--reader-config`, `--reader-property`. Starsze `--producer.config`, `--producer-property` i `--property` są oznaczone jako deprecated w bieżącym help Confluent. Inne ważne flagi: `--compression-codec`, `--request-required-acks`, `--batch-size`, `--sync`. Źródło: [Confluent — kafka-console-producer.sh](#).

4.2. kafka-console-consumer.sh

```
kafka-console-consumer.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config /etc/kafka/client.properties \
  --topic orders \
  --from-beginning \
  --formatter-property print.key=true \
  --formatter-property print.partition=true \
  --formatter-property print.offset=true \
  --max-messages 100
```

Odczyt konkretnej partycji i offsetu:

```
kafka-console-consumer.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config /etc/kafka/client.properties \
  --topic orders \
  --partition 0 \
  --offset 500
```

Odczyt transakcyjny:

```
kafka-console-consumer.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --topic orders \
  --group debug-orders \
  --isolation-level read_committed
```

W 4.3 preferowane są `--command-config`, `--command-property`, `--formatter-config`, `--formatter-property`. Starsze `--consumer.config`, `--consumer-property` i `--property` są deprecated. Źródło: [Confluent — kafka-console-consumer.sh](#).

4.3. kafka-consumer-groups.sh --list

```
kafka-consumer-groups.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" \
  --list

kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \
  --list --state stable,empty

kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \
  --list --type classic,consumer
```

Źródło: [Apache Kafka 4.3 — Managing Consumer Groups](#).

4.4. --describe

```
# Offsety i lag
kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" --describe --group orders-cg

# Członkowie i assignment
kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \
  --describe --group orders-cg --members --verbose

# Stan grupy
kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \
  --describe --group orders-cg --state
```

Źródło: [Apache Kafka 4.3 — Consumer group details](#).

4.5. --reset-offsets

Grupa powinna być nieaktywna. Najpierw dry-run, potem ten sam zakres z --execute:

```
kafka-consumer-groups.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" \
  --reset-offsets \
  --group orders-cg \
  --topic orders \
  --to-earliest \
  --dry-run

kafka-consumer-groups.sh \
  --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" \
  --reset-offsets \
  --group orders-cg \
  --topic orders \
```

```
--to-earliest \  
--execute
```

Inne specyfikacje resetu (wybierz jedną):

```
--to-latest  
--to-current  
--to-offset 12345  
--shift-by -1000  
--to-datetime '2026-09-07T08:00:00.000'  
--by-duration 'PT6H'  
--from-file offsets.csv
```

Zakres: `--topic topic:0,1,2`, wielokrotne `--topic` albo `--all-topics`. `--export` zwraca plan CSV. Źródło: [Apache Kafka 4.3 — Resetting Consumer Group Offsets](#).

4.6. Usuwanie grupy lub wybranych offsetów

```
# Cała nieaktywna grupa  
kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --delete --group orders-cg  
  
# Offsety grupy tylko dla topicu  
kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --delete-offsets --group orders-cg --topic orders
```

Źródło: [Apache Kafka 4.3 — Deleting Consumer Groups](#).

5. ACL i bezpieczeństwo

5.1. `kafka-acls.sh --add`

Nadanie producentowi prawa zapisu:

```
kafka-acls.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --add \  
  --allow-principal 'User:alice' \  
  --allow-host '*' \  
  --producer \  
  --topic orders
```

Konsument topicu w konkretnej grupie:


```
kafka-acls.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --add \  
  --allow-principal 'User:alice' \  
  --consumer \  
  --topic orders \  
  --group orders-cg
```

Jawne operacje i prefiks:

```
kafka-acls.sh --bootstrap-server "$BOOTSTRAP" \  
  --add \  
  --allow-principal 'User:alice' \  
  --operation Read \  
  --operation Describe \  
  --topic 'orders-' \  
  --resource-pattern-type prefixed
```

Skróty `--producer` i `--consumer` rozwijają typowe zestawy operacji. Dokładna semantyka i pełne opcje są w [Apache Kafka 4.3 — Authorization and ACLs](#).

5.2. Listowanie ACL

```
kafka-acls.sh --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --list  
  
kafka-acls.sh --bootstrap-server "$BOOTSTRAP" \  
  --list --topic orders  
  
kafka-acls.sh --bootstrap-server "$BOOTSTRAP" \  
  --list --topic 'orders-2026' --resource-pattern-type match
```

`match` znajduje pasujące wpisy literal/prefixed/wildcard. Źródło: [Apache Kafka 4.3 — Listing ACLs](#).

5.3. Usuwanie ACL

```
kafka-acls.sh --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --remove \  
  --allow-principal 'User:alice' \  
  --operation Read \  
  --operation Describe \  
  --topic 'orders-' \  
  --resource-pattern-type prefixed
```

Domyślnie narzędzie prosi o potwierdzenie; użycie opcji omijającej prompt należy poprzedzić `--list` z identycznym filtrem. Źródło: [Apache Kafka 4.3 — Removing ACLs](#).

5.4. Klient TLS

Minimalny /etc/kafka/client-ssl.properties bez mTLS:

```
security.protocol=SSL
ssl.truststore.location=/etc/kafka/secrets/client.truststore.jks
ssl.truststore.password=changeit
```

Z mTLS:

```
security.protocol=SSL
ssl.truststore.location=/etc/kafka/secrets/client.truststore.jks
ssl.truststore.password=changeit
ssl.keystore.location=/etc/kafka/secrets/client.keystore.jks
ssl.keystore.password=changeit
ssl.key.password=changeit
```

Użycie:

```
kafka-topics.sh --bootstrap-server 'broker1:9093' \
  --command-config /etc/kafka/client-ssl.properties --list
```

Weryfikacja hostname jest domyślnie włączona; nie wyłączaj

ssl.endpoint.identification.algorithm w produkcji bez uzasadnienia. Źródło: [Apache Kafka 4.3 — Encryption and Authentication using SSL](#).

5.5. Klient SASL/SCRAM przez TLS

/etc/kafka/client-sasl.properties:

```
security.protocol=SASL_SSL
sasl.mechanism=SCRAM-SHA-512
sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule required use
ssl.truststore.location=/etc/kafka/secrets/client.truststore.jks
ssl.truststore.password=changeit
```

```
kafka-console-producer.sh \
  --bootstrap-server 'broker1:9094' \
  --command-config /etc/kafka/client-sasl.properties \
  --topic orders
```

Kafka 4.3 obsługuje GSSAPI, PLAIN, SCRAM-SHA-256, SCRAM-SHA-512 i OAUTHBEARER.

PLAIN/SCRAM powinny korzystać z TLS (SASL_SSL). Źródło: [Apache Kafka 4.3 — Authentication using SASL](#).

6. Replikacja, liderzy i katalogi logów

6.1. kafka-leader-election.sh

Preferowany lider dla wszystkich kwalifikujących się partycji:

```
kafka-leader-election.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --election-type preferred \  
  --all-topic-partitions
```

Jedna partycja:

```
kafka-leader-election.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --election-type preferred \  
  --topic orders \  
  --partition 0
```

Z pliku:

```
{"partitions":[{"topic":"orders","partition":0},{"topic":"payments","partition":2}]}
```

```
kafka-leader-election.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --election-type unclean \  
  --path-to-json-file partitions.json
```

unclean może spowodować utratę danych; używaj tylko świadomie, gdy partycja nie ma lidera.

Źródła: [Apache Kafka 4.3 — Balancing leadership](#) i [Confluent — kafka-leader-election.sh](#).

6.2. kafka-log-dirs.sh

```
# Wszystkie brokery i topiki  
kafka-log-dirs.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG" \  
  --describe  
  
# Filtry  
kafka-log-dirs.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --describe \  
  --broker-list '1,2' \  
  --topic-list 'orders,payments'
```

Narzędzie pokazuje użycie katalogów logów i repliki przypisane do katalogów; przydatne przed/po reassignment. Źródło: [Confluent — kafka-log-dirs.sh](#).

6.3. Zmiana replication factor

Nie ma bezpiecznej komendy `kafka-topics --alter --replication-factor`. Zmień listę replicas każdej partycji przez `kafka-reassign-partitions --execute`, a potem sprawdź:

```
kafka-reassign-partitions.sh --bootstrap-server "$BOOTSTRAP" \  
  --reassignment-json-file increase-rf.json --execute  
  
kafka-reassign-partitions.sh --bootstrap-server "$BOOTSTRAP" \  
  --reassignment-json-file increase-rf.json --verify  
  
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \  
  --describe --topic orders
```

Źródło: [Apache Kafka 4.3 — Increasing replication factor](#).

7. Monitoring i diagnostyka

7.1. kafka-broker-api-versions.sh

```
# Wersja narzędzia  
kafka-broker-api-versions.sh --version  
  
# API obsługiwane przez brokerów  
kafka-broker-api-versions.sh \  
  --bootstrap-server "$BOOTSTRAP" \  
  --command-config "$ADMIN_CFG"
```

Druga forma negocjuje i pokazuje zakres wersji każdego Kafka API dla osiągalnych brokerów; jest użyteczna w diagnostyce zgodności klient-broker. Źródło: [Confluent — kafka-broker-api-versions.sh](#).

7.2. kafka-run-class.sh

Cel: ogólny launcher klas z classpath dystrybucji Kafka. Preferuj dedykowane wrappery (`kafka-dump-log.sh`, `kafka-get-offsets.sh`, `kafka-producer-perf-test.sh`), ponieważ nazwy klas i ich stabilność nie są publicznym API.

Ogólny wzorzec:

```
kafka-run-class.sh <pełna.nazwa.Klasy> [opcje-klasy]
```

Przykład niskopoziomowego uruchomienia dump tool:

```
kafka-run-class.sh kafka.tools.DumpLogSegments \  
  --files /var/lib/kafka/data/orders-0/00000000000000000000.log
```

W CP/Kafka 4.3 najpierw użyj równoważnego, stabilniejszego wrappera:

```
kafka-dump-log.sh \  
  --files /var/lib/kafka/data/orders-0/00000000000000000000.log
```

Przykładowe wspierane diagnostyczne wrappery:

```
kafka-get-offsets.sh --bootstrap-server "$BOOTSTRAP" --topic orders  
kafka-producer-perf-test.sh --topic perf \  
  --num-records 100000 --record-size 1000 --throughput -1 \  
  --bootstrap-server "$BOOTSTRAP"  
kafka-consumer-perf-test.sh --bootstrap-server "$BOOTSTRAP" \  
  --topic perf --num-records 100000
```

Dokumentacja poleceń wrapperów: [Confluent — Kafka CLI Tools](#). Implementacja DumpLogSegments i jej help: [Apache Kafka source — DumpLogSegments.scala](#). Ta ostatnia strona śledzi trunk, nie tag 4.3.0, dlatego lokalne --help jest obowiązkowe przed automatyzacją.

7.3. Włączenie JMX

Kafka wyłącza remote JMX domyślnie. Dla procesu uruchamianego skrypcem:

```
export JMX_PORT=9999  
export KAFKA_JMX_OPTS='-Dcom.sun.management.jmxremote=true \  
-Dcom.sun.management.jmxremote.authenticate=true \  
-Dcom.sun.management.jmxremote.ssl=true'  
  
kafka-server-start.sh /etc/kafka/server.properties
```

Nie wystawiaj niezabezpieczonego JMX w produkcji. Źródło: [Apache Kafka 4.3 — Monitoring, JMX security](#).

7.4. Kluczowe kafka.server:type=raft-metrics

Metryka	Interpretacja
current-state	leader, candidate, voted, follower, unattached, observer
current-leader	ID lidera; -1 = nieznan
current-voted	ID, na które oddano głos; -1 = brak
current-epoch	aktualny epoch quorum
high-watermark	zatwierdzona granica logu; -1 = nieznan
log-end-offset	lokalny koniec Raft logu

Metryka	Interpretacja
commit-latency-avg/max	średnie/maksymalne opóźnienie commit
election-latency-avg/max	średnie/maksymalne opóźnienie elekcji
fetch-records-rate	tempo rekordów pobieranych przez followera
append-records-rate	tempo appendów lidera
poll-idle-ratio-avg	bezczynność wątku Raft I/O

Najważniejsze alerty operacyjne: nieznany lider, kandydowanie przez dłuższy czas, rosnąca różnica LEO-HW, rosnący follower lag i wzrost commit/election latency. Nazwy MBean i opisy: [Apache Kafka 4.3 — KRaft monitoring metrics](#). Confluent publikuje również własny katalog: [Confluent — Kafka Broker Metrics](#).

8. Narzędzia Confluent Platform 8.3

8.1. Confluent CLI — wybór kontekstu

Confluent CLI to oddzielny produkt z własnym cyklem wydań. CP 8.3.x wspiera linię CLI od 4.61.0 do najnowszej 4.x, dlatego składnia `confluent-cli/current` może być nowsza niż konkretny klient zainstalowany wraz z CP 8.3.0. Sprawdź `confluent version` i lokalny help. ([Confluent — versions/interoperability](#))

```
confluent version
confluent context list
confluent context use <context-name>
```

Referencja: [Confluent CLI Command Reference](#).

8.2. `confluent kafka topic`

Tworzenie:

```
confluent kafka topic create orders \
  --url 'https://rest-proxy.example.com:8082' \
  --partitions 12 \
  --replication-factor 3 \
  --config 'cleanup.policy=delete,min.insync.replicas=2' \
  --if-not-exists
```

Listowanie i opis:

```
confluent kafka topic list \
  --url 'https://rest-proxy.example.com:8082'
```

```
confluent kafka topic describe orders \
  --url 'https://rest-proxy.example.com:8082'
```

Aktualizacja konfiguracji i usunięcie:

```
confluent kafka topic update orders \
  --url 'https://rest-proxy.example.com:8082' \
  --config 'retention.ms=1209600000'

# W skrypcie nieinteraktywnym --force pomija pytanie o potwierdzenie.
confluent kafka topic delete orders \
  --url 'https://rest-proxy.example.com:8082' \
  --force
```

Produce/consume:

```
confluent kafka topic produce orders

confluent kafka topic consume orders \
  --from-beginning \
  --group cli-debug \
  --print-key \
  --print-offset
```

W CP polecenia administracyjne `confluent kafka topic ...` korzystają z Kafka REST/REST Proxy i mogą wymagać `--url`, mTLS (`--certificate-authority-path`, `--client-cert-path`, `--client-key-path`) lub kontekstu. Opcje różnią się od trybu Confluent Cloud. Źródła: [Confluent CLI — topic index](#), [topic create](#), [topic list](#), [topic describe](#), [topic update](#) w CP, [topic delete](#), [topic produce](#) i [topic consume](#).

8.3. confluent cluster — rejestr klastrów CP w MDS

Nie myl z `kafka-cluster.sh` ani cloudowym `confluent kafka cluster`. `confluent cluster` zarządza wpisami Cluster Registry w Metadata Service (MDS).

```
confluent login --url 'https://mds.example.com:8090'

confluent cluster register \
  --cluster-name prod-eu \
  --kafka-cluster '<kafka-cluster-id>' \
  --hosts 'broker1:9094,broker2:9094' \
  --protocol SASL_SSL

confluent cluster list

confluent cluster describe \
  --url 'https://mds.example.com:8090'

confluent cluster unregister \
  --cluster-name prod-eu
```

Źródła: [Confluent — Cluster Registry](#), [CLI cluster register](#), [CLI cluster list](#) i [CLI cluster describe](#).

8.4. confluent kafka cluster — Confluent Cloud

Ta gałąź dotyczy przede wszystkim klastrów Confluent Cloud, nie lokalnego brokera CP:

```
confluent kafka cluster create demo \  
  --cloud aws \  
  --region eu-central-1 \  
  --type basic  
  
confluent kafka cluster use <lkc-id>
```

Źródła: [Confluent CLI — kafka cluster create](#) i [kafka cluster use](#).

8.5. Schema Registry — Confluent CLI

Rejestracja i odczyt:

```
confluent schema-registry schema create \  
  --schema-registry-endpoint 'https://sr.example.com:8081' \  
  --subject orders-value \  
  --schema orders.avsc \  
  --type avro  
  
confluent schema-registry schema list \  
  --schema-registry-endpoint 'https://sr.example.com:8081' \  
  --subject-prefix orders  
  
confluent schema-registry schema describe \  
  --schema-registry-endpoint 'https://sr.example.com:8081' \  
  --subject orders-value \  
  --version latest
```

Przy prywatnym CA/mTLS dodaj `--certificate-authority-path`, `--client-cert-path`, `--client-key-path`. Składnia bieżącego Confluent CLI: [schema create](#), [schema list](#), [schema describe](#).

8.6. Schema Registry — REST

Podstawowe zmienne:

```
export SR='https://sr.example.com:8081'  
export SR_AUTH='user:password'
```

Health/subjects/wersje:

```
curl -sS -u "$SR_AUTH" "$SR/subjects" | jq .  
curl -sS -u "$SR_AUTH" "$SR/subjects/orders-value/versions" | jq .
```



```
curl -sS -u "$SR_AUTH" "$SR/subjects/orders-value/versions/latest" | jq .
curl -sS -u "$SR_AUTH" "$SR/schemas/ids/123" | jq .
```

Rejestracja Avro:

```
jq -n --rawfile schema orders.avsc \
  '{schemaType:"AVRO",schema:$schema}' > register.json

curl -sS -u "$SR_AUTH" \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -X POST \
  --data @register.json \
  "$SR/subjects/orders-value/versions" | jq .
```

Test zgodności:

```
curl -sS -u "$SR_AUTH" \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -X POST \
  --data @register.json \
  "$SR/compatibility/subjects/orders-value/versions/latest" | jq .
```

Soft-delete, potem opcjonalny hard-delete:

```
curl -sS -u "$SR_AUTH" -X DELETE \
  "$SR/subjects/orders-value" | jq .

curl -sS -u "$SR_AUTH" -X DELETE \
  "$SR/subjects/orders-value?permanent=true" | jq .
```

Źródło endpointów i modeli: [Confluent Platform — Schema Registry API Reference](#). Przykładowe workflow on-prem: [Schema Registry tutorial](#).

8.7. Control Center

Control Center jest webowym narzędziem obserwacji i administracji, a nie zamiennikiem podstawowych CLI Kafka. Dla CP 8.3 użyj zgodnej, niezależnie wersjonowanej linii Control Center wskazanej w macierzy zgodności. Źródła: [Control Center Overview](#) i [Confluent — versions/interoperability](#).

Komendy `confluent local services control-center ...` służą lokalnemu środowisku developerskiemu, nie produkcyjnemu klastrówi:

```
confluent local services control-center status
confluent local services control-center start
confluent local services control-center stop
```

Referencja lokalnej gałęzi CLI: [Confluent CLI — local Control Center](#).

8.8. MirrorMaker 1 vs MirrorMaker 2

Stary `kafka-mirror-maker.sh` jest deprecated. Dla nowego wdrożenia użyj MirrorMaker 2 na Kafka Connect:

```
connect-mirror-maker.sh /etc/kafka/mm2.properties

# Opcjonalnie uruchom tylko wskazane klastry docelowe:
connect-mirror-maker.sh \
  --clusters secondary \
  /etc/kafka/mm2.properties
```

Minimalny `mm2.properties`:

```
clusters=primary,secondary

primary.bootstrap.servers=primary-1:9092,primary-2:9092
secondary.bootstrap.servers=secondary-1:9092,secondary-2:9092

primary->secondary.enabled=true
primary->secondary.topics=orders|payments

replication.factor=3
checkpoints.topic.replication.factor=3
heartbeats.topic.replication.factor=3
offset-syncs.topic.replication.factor=3

sync.topic.acls.enabled=false
emit.heartbeats.enabled=true
emit.checkpoints.enabled=true
```

Źródła: [Apache Kafka 4.3 — Geo-Replication / Cross-Cluster Data Mirroring](#) i [Confluent — connect-mirror-maker.sh](#).

9. Co jest specyficzne dla Kafka 4.3.x / CP 8.3

1. **ZooKeeper nie istnieje w tej linii.** Kafka 4.x wspiera wyłącznie KRaft; w CP ZooKeeper został usunięty od 8.0. Klaster ZooKeeper trzeba zmigrować w starszej wspieranej linii przed wejściem na 8.3. ([Apache Kafka 4.3 — Upgrade](#), [Confluent — ZooKeeper compatibility](#))
2. `kraft.version` **jest oddzielnym feature level.** Poziom 1 włącza dynamiczny membership kontrolerów; 0 lub brak oznacza statyczny voter set. ([Apache Kafka 4.3 — Static versus Dynamic KRaft Quorums](#))
3. **Dynamiczne quorum używa** `controller.quorum.bootstrap.servers`, **nie** `controller.quorum.voters`. Bootstrap to lista discovery, a nie definicja aktualnego voter setu. ([Apache Kafka 4.3 — KRaft configuration](#))
4. **Nowy klaster:** preferowany bootstrap jednego kontrolera przez `kafka-storage format --standalone`, potem `--no-initial-controllers + observer catch-up + add-controller`. ([Apache Kafka 4.3 — Provisioning Nodes](#))

5. **Finalizacja 4.3:** `kafka-features ... upgrade --release-version 4.3` ustawia `metadata.version=4.3-IV0` i powiązane feature levels dopiero po ukończeniu rolling upgrade. Downgrade metadanych może być niemożliwy. ([Confluent — Upgrade CP](#))
6. **Zmiany nazw opcji konsolowych:** bieżący help 4.3 preferuje `--command-config`, `--command-property`, `--reader-property` i `--formatter-property`; starsze `--producer.config`, `--consumer.config` i `--property` są deprecated. ([Confluent — Kafka CLI Tools](#))
7. `kafka-replica-verification` i **MirrorMaker 1** są deprecated. W nowych wdrożeniach stosuj standardowe metryki/reassignment verification oraz MirrorMaker 2. ([Confluent — Kafka CLI Tools](#))
8. `platform/current` **nie** znaczy „bit po bicie CP 8.3.0”. Strona może uwzględniać późniejsze patche 8.3.x. Dla skryptów automatyzujących destrukcyjne operacje zachowaj wynik lokalnego `--help` wraz z `--version`.

10. Szybka checklista bezpiecznej administracji

```
# 1. Wersja
kafka-topics.sh --version

# 2. Quorum i feature levels
kafka-metadata-quorum.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" describe --status
kafka-features.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" describe

# 3. Partycje bez lidera / niedoreplikowane / poniżej min ISR
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" --describe --unavailable-partitions
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" --describe --under-replicated-partitions
kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" --describe --under-min-isr-partitions

# 4. Consumer lag
kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" --describe --all-groups

# 5. Aktywne reassignments i log dirs
kafka-reassign-partitions.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" --list
kafka-log-dirs.sh --bootstrap-server "$BOOTSTRAP" \
  --command-config "$ADMIN_CFG" --describe
```

Wszelkie `format`, `force-standalone`, `unclean`, `delete`, `reset-offsets --execute`, `ACL remove`, `hard-delete` Schema Registry oraz ręczny reassignment wymagają planu rollback/backup i odnotowania faktycznej wersji narzędzia.

Źródła

Pełne URL-e wszystkich dokumentów cytowanych w ściadze:

1. Apache Kafka 4.3 — Basic Kafka Operations
<https://kafka.apache.org/43/operations/basic-kafka-operations/>
2. Apache Kafka 4.3 — KRaft
<https://kafka.apache.org/43/operations/kraft/>
3. Apache Kafka 4.3 — Monitoring
<https://kafka.apache.org/43/operations/monitoring/>
4. Apache Kafka 4.3 — Upgrade
<https://kafka.apache.org/43/getting-started/upgrade/>
5. Apache Kafka 4.3 — Authorization and ACLs
<https://kafka.apache.org/43/security/authorization-and-acls/>
6. Apache Kafka 4.3 — Authentication using SASL
<https://kafka.apache.org/43/security/authentication-using-sasl/>
7. Apache Kafka 4.3 — Encryption and Authentication using SSL
<https://kafka.apache.org/43/security/encryption-and-authentication-using-ssl/>
8. Apache Kafka 4.3 — Geo-Replication / Cross-Cluster Data Mirroring
<https://kafka.apache.org/43/operations/geo-replication-cross-cluster-data-mirroring/>
9. Apache Kafka source — DumpLogSegments.scala (trunk, nie tag 4.3.0)
<https://github.com/apache/kafka/blob/trunk/core/src/main/scala/kafka/tools/DumpLogSegments.scala>
10. Confluent — Kafka Command-Line Interface Tools
<https://docs.confluent.io/kafka/operations-tools/kafka-tools.html>
11. Confluent — Topic Operations
<https://docs.confluent.io/kafka/operations-tools/topic-operations.html>
12. Confluent Platform 8.3 — Release Notes
<https://docs.confluent.io/platform/current/release-notes/index.html>
13. Confluent — Supported Versions and Interoperability
<https://docs.confluent.io/platform/current/installation/versions-interoperability.html>
14. Confluent — Upgrade Confluent Platform
<https://docs.confluent.io/platform/current/installation/upgrade.html>
15. Confluent — Configure and Monitor KRaft
<https://docs.confluent.io/platform/current/kafka-metadata/config-kraft.html>
16. Confluent — Disaster Recovery for Multi-Region KRaft Clusters
<https://docs.confluent.io/operator/current/co-disaster-recovery.html>
17. Confluent — Broker Configuration Reference
<https://docs.confluent.io/platform/current/installation/configuration/broker-configs.html>
18. Confluent — Kafka Broker Metrics
<https://docs.confluent.io/platform/current/kafka/broker-metrics.html>

19. Confluent CLI — Command Reference

<https://docs.confluent.io/confluent-cli/current/command-reference/overview.html>

20. Confluent CLI — kafka topic

<https://docs.confluent.io/confluent-cli/current/command-reference/kafka/topic/index.html>

21. Confluent CLI — kafka topic create

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/topic/confluent_kafka_topic_create.html

22. Confluent CLI — kafka topic list

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/topic/confluent_kafka_topic_list.html

23. Confluent CLI — kafka topic describe

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/topic/confluent_kafka_topic_describe.html

24. Confluent Platform — zarządzanie topicami przez Confluent CLI

<https://docs.confluent.io/platform/current/kafka/manage-topics.html>

25. Confluent CLI — kafka topic delete

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/topic/confluent_kafka_topic_delete.html

26. Confluent CLI — kafka topic produce

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/topic/confluent_kafka_topic_produce.html

27. Confluent CLI — kafka topic consume

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/topic/confluent_kafka_topic_consume.html

28. Confluent CLI — cluster register

https://docs.confluent.io/confluent-cli/current/command-reference/cluster/confluent_cluster_register.html

29. Confluent CLI — cluster list

https://docs.confluent.io/confluent-cli/current/command-reference/cluster/confluent_cluster_list.html

30. Confluent CLI — cluster describe

https://docs.confluent.io/confluent-cli/current/command-reference/cluster/confluent_cluster_describe.html

31. Confluent Platform — Cluster Registry

<https://docs.confluent.io/platform/current/security/cluster-registry.html>

32. Confluent CLI — kafka cluster create

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/cluster/confluent_kafka_cluster_create.html

33. Confluent CLI — kafka cluster use

https://docs.confluent.io/confluent-cli/current/command-reference/kafka/cluster/confluent_kafka_cluster_use.html

34. Confluent CLI — Schema Registry schema create
https://docs.confluent.io/confluent-cli/current/command-reference/schema-registry/schema/confluent_schema-registry_schema_create.html
35. Confluent CLI — Schema Registry schema list
https://docs.confluent.io/confluent-cli/current/command-reference/schema-registry/schema/confluent_schema-registry_schema_list.html
36. Confluent CLI — Schema Registry schema describe
https://docs.confluent.io/confluent-cli/current/command-reference/schema-registry/schema/confluent_schema-registry_schema_describe.html
37. Confluent Platform — Schema Registry REST API
<https://docs.confluent.io/platform/current/schema-registry/develop/api.html>
38. Confluent Platform — Schema Registry on-prem tutorial
https://docs.confluent.io/platform/current/schema-registry/schema_registry_onprem_tutorial.html
39. Confluent Control Center — Overview
<https://docs.confluent.io/control-center/current/overview.html>
40. Confluent CLI — local Control Center
<https://docs.confluent.io/confluent-cli/current/command-reference/local/services/control-center/index.html>